

Numerical Analysis presentation

Team_7

主讲人：王玉佩，吴旭文

Question 1

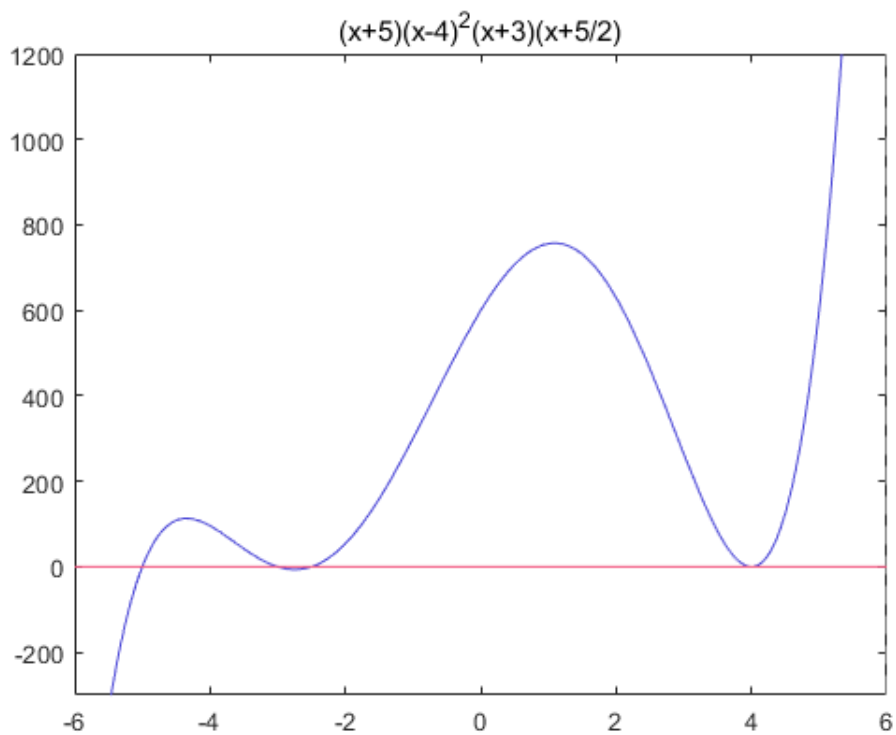
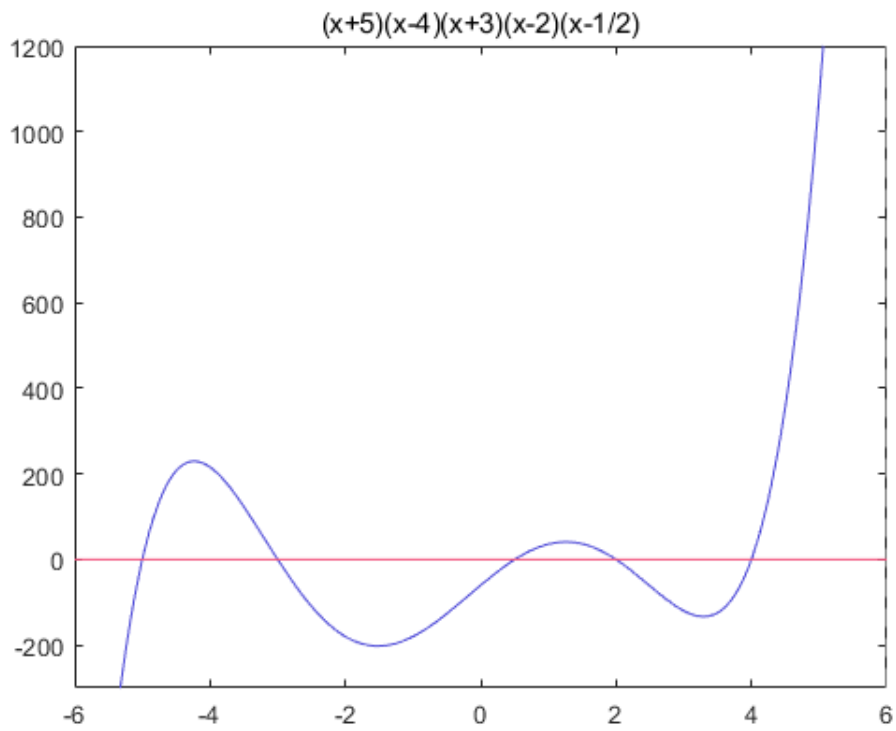
1.1

Design two nonlinear algebraic equations (degree at least 5 , with no less than 3 zeros) and find all their zeros (error less than 0.001).

So,we firstly design the quintic algebraic equations as follows.

$$y_1 = (x + 5)(x - 4)(x + 3)(x - 2)(x - \frac{1}{2})$$
$$y_2 = (x + 5)(x - 4)^2(x + 3)(x + \frac{5}{2})$$

we use the Matlab to print the figure of the equation.



1.2

By analyzing the y_1 images, we can easily the minimum distance of zero is less than $\frac{1}{2}$, and the minimum distance of zero in y_2 is less than $\frac{1}{2}$ too. Thus, We divide the whole interval $[-6, 6]$ into a number of cells with a length of $1/2$, and using the method of bisection solve for zero separately.

```

1 | x0 = zeros(1, 5);
2 | k = 0;

```

```

3  L = 1/2;%分割长度
4  x0 = unique(x0);
5  for i = -6:L:6
6      if (y(i)*y(i+L)<=0)
7          k = k + 1;
8          x0(k) = f_Bis(y, i, i+L, 0.001);%k为第k的零点
9      end
10 end
11 fprintf("零点的横坐标为:")
12
13 %二分法具体函数
14 function z=f_Bis(f, a, b, accurate)
15 p=(a+b)/2;
16 n=0;
17 z=0;
18 while (abs(f(p)-0)>accurate)
19     if (f(p)==0)%判断端点值是否为零点
20         z=p;
21         break;
22     end
23     if (f(a)==0)
24         z=a;
25         break;
26     end
27     if (f(b)==0)
28         z=b;
29         break;
30     end
31     if (f(a)*f(p)<0)
32         b=p;
33         p=(a+b)/2;
34     else
35         a=p;
36         p=(a+b)/2;
37     end
38     z=p;
39     n=n+1;
40 end
41 n;
42 end

```

Because of the existence of multiple zeros, Therefore, when solving the zeros, we should also consider the order of the zeros.

We take the derivative of the function, we get the first derivative of the function, and then we determine whether the zero at the first derivative is zero, and if it's zero, then it's the second zero, and so on.

函数的一阶导数值：

$dy_x =$

$$\left(405 \quad -49 \quad \frac{845}{16} \quad 0\right)$$

函数的二阶导数值：

$$ddy_x = (-909 \quad 175 \quad 221 \quad 819)$$

- -

<i>zeros</i>	-5.0000	-3.0000	-2.5000	4.0000
<i>order</i>	1.0000	1.0000	1.0000	2.0000

```

1 function B = Judge(y, x0)
2 [C,R] = size(x0);
3 k = 1;
4 syms x
5 y1 = (x+5)*(x-4)*(x+3)*(x-2)*(x-1/2);
6 y=(x+5)*(x-4)^2*(x+3)*(x+5/2);%该函数只能键入，用参数输入时无法识别
7 f = y;
8 dy = diff(f)%求一阶导
9 ddy = diff(f,2);求二阶导
10 fprintf("函数的一阶导数值:")
11 dy_x = subs(dy, x, x0)
12 dy_x =double(dy_x);%强制转换
13 fprintf("函数的二阶导数值:")
14 ddy_x = subs(ddy, x, x0)
15 ddy_x = double(ddy_x);
16 for i=1:R
17     if(dy_x(i) == 0)
18         x0(2,k) = 2;
19         if(ddy_x(i) == 0)
20             x0(2,k) = 3;
21         end
22     else%此处只判断到三阶零点
23         x0(2,k) = 1;
24     end
25     k = k + 1;
26 end

```

```
27 | B = x0;  
28 | end
```

Question 2

Find the numerical solution of the following nonlinear equations using iteration method and Newton method respectively and show your analysis of error by using norms of vector and matrix and demonstrate your computer code

$$\begin{cases} x_1^2 - 10x_1 + x_2^2 + 8 = 0 \\ x_1x_2^2 + x_1 - 12x_2 + 7 = 0 \end{cases}$$

2.1

Fixed-point iteration method:

We have the following system of equations $\begin{cases} y_1 = f_1(x_1, x_2) \\ y_2 = f_2(x_1, x_2) \end{cases}$, we set the initial point $P_0 = \begin{pmatrix} x_0 \\ y_0 \end{pmatrix}$, the iterative formula is $\begin{cases} x_1^{(n+1)} = f_1(x_1^{(n)}, x_2^{(n)}) \\ x_2^{(n+1)} = f_2(x_1^{(n)}, x_2^{(n)}) \end{cases}$.

In order to compare with the Newton-Raphson iteration method, The error is

$$\left\| \begin{bmatrix} x_1^{(n+1)} \\ x_2^{(n+1)} \end{bmatrix} - \begin{bmatrix} x_1^{(n)} \\ x_2^{(n)} \end{bmatrix} \right\| = \left\| \begin{bmatrix} f_1(x_1^n, x_2^n) - f_1(x_1^{(n-1)}, x_2^{(n-1)}) \\ f_2(x_1^n, x_2^n) - f_2(x_1^{(n-1)}, x_2^{(n-1)}) \end{bmatrix} \right\|$$

```
1 | N = 100;% 设置最大迭代次数  
2 | epsilon = 1e-9;% 设置迭代的精度  
3 | %Fixed-point iteration  
4 | x(1,:) = [1;1];%迭代初始点  
5 | for i = 1:N  
6 |     x(i+1,1) = 1/10*(x(i,1)^2+x(i,2)^2+8);  
7 |     x(i+1,2) = 1/12*(x(i,1)+x(i,1)*x(i,2)^2+7);  
8 |     dx_Fp(i,:) = x(i+1,:) - x(i,:);  
9 |     if(sum(dx_Fp(i,:).^2) < epsilon)  
10 |         break;  
11 |     end  
12 | end  
13 | x_Fp = x
```

Newton-Raphson Iteration

Firstly, we set the initial point $x_0 = \begin{pmatrix} x_0 \\ y_0 \end{pmatrix}$, the $x^{(k+1)} - x^{(k)} = \frac{-F(x^{(k)})}{J(x^{(k)})}$.

we set the $d^{(k)} = x^{(k+1)} - x^{(k)}$.

Secondly, The termination condition of the iteration is $\|d^{(k)}\| < \varepsilon$

```
1 %Newton-Raphson iteration
2 syms x1 x2;
3 f1(x1,x2) = x1^2 - 10*x1 + x2^2 + 8;
4 f2(x1,x2) = x1*x2^2 + x1 - 12*x2 + 7;
5
6 F(x1,x2) = [f1;f2];
7
8 x_solve(:,1) = [1;1];%迭代初始点
9
10 J(x1,x2) = jacobian(F, [x1 x2]);
11
12 dx_Nr =[0;0];          % X(n+1)-X(n) = d(X); 第一行表示第一次迭代X(x1,x2)
    值与前一值的插值
13 for i = 1:N              % J*d(x) = -F(X) => d(x) = J \ -F(x)
14     dx_Nr(:,i) = J(x_solve(1,i),x_solve(2,i))\(-
    F(x_solve(1,i),x_solve(2,i)));
15     if(sqrt(sum(dx_Nr(:,i).^2)) < epsilon) %
16         x_solve(:,i+1) = x_solve(:,i) + dx_Nr(:,i);
17         break;
18     else
19         x_solve(:,i+1) = x_solve(:,i) + dx_Nr(:,i);
20     end
21 end
22 dx_Nr = dx_Nr';
23 x_Nr = x_solve'
```

Result:

1	x_Fp = 10×2		
2	1.0000	1.0000	%误差
3	1.0000	0.7500	0.0625
4	0.9563	0.7135	0.0032
5	0.9424	0.7036	2.9203e-04
6	0.9383	0.7007	2.4534e-05
7	0.9371	0.6999	2.0193e-06
8	0.9368	0.6997	1.6524e-07

```

9         0.9367      0.6996      1.3499e-08
10        0.9367      0.6996      1.1022e-09
11        0.9367      0.6996      8.9990e-11
12        %迭代 9次
13
14        x_Nr = 6×2
15
16        1.0000      1.0000
17        0.9211      0.6842      0.3255
18        0.9366      0.6995      0.0218
19        0.9367      0.6996      9.2006e-05
20        0.9367      0.6996      1.6056e-09
21        0.9367      0.6996      5.7481e-17
22        %迭代 5次
23        % it is easy to find that the Newton method is better than fix-point
        method

```

We generalize the two-dimensional model to the three-dimensional model,

$$\begin{cases} 5x_1 - x_2^2 + x_1x_3 = 2 \\ x_1x_3 + 4x_2 - x_3 = 0 \\ x_1^2 + x_2 - 3x_3 = 1 \end{cases}$$

we only need to expand a variable, The changed details are as follows:

```

1        %Fixed-point iteration
2        x(1,1) = 1;x(1,2) = 1;x(1,3) =1;%迭代初始点
3        for i = 1:N
4            x(i+1,1) = 1/5*(x(i,2)^2-x(i,1)*x(i,3)+2);
5            x(i+1,2) = 1/4*(x(i,3)-x(i,1)*x(i,3));
6            x(i+1,3) = 1/3*(x(i,1)^2+x(i,2)-1);
7            dx_Fp(i,:) = x(i+1,:) - x(i,:);
8            if(sum(dx_Fp(i,:).^2)< epsilon)
9                break;
10           end
11       end
12       x_Fp = x
13
14
15       %Newton-Raphson iteration
16       syms x1 x2 x3;
17       f1(x1,x2,x3) = 5*x1-x2^2+x1*x3-2;
18       f2(x1,x2,x3) = x1*x3+4*x2-x3;
19       f3(x1,x2,x3) = x1^2+x2-3*x3-1;
20

```

```

21 F(x1, x2, x3) = [f1;f2;f3];
22
23 x_solve(:, 1) = [0;0;0];%迭代初始点
24
25 J(x1, x2, x3) = jacobian(F, [x1 x2 x3]);
26
27 dx_Nr =[1;1;1];          % X(n+1)-X(n) = d(X); 第一行表示第一次迭代
                             X(x1, x2) 值与前一值的差值
28 for i = 1:N                % J*d(x) = -F(X) => d(x) = J \ -F(x)
29     dx_Nr(:, i) = J(x_solve(1, i), x_solve(2, i), x_solve(3, i)) \ (-
F(x_solve(1, i), x_solve(2, i), x_solve(3, i)));
30     if(sqrt(sum(dx_Nr(:, i).^2)) < epsilon)
31         x_solve(:, i+1) = x_solve(:, i) + dx_Nr(:, i);
32         break;
33     else
34         x_solve(:, i+1) = x_solve(:, i) + dx_Nr(:, i);
35     end
36 end
37 dx_Nr = dx_Nr';
38 x_Nr = x_solve'

```

Question 3

3.1

(1)Generate matrix $A(50 \times 50)$ and vector $b(50 \times 1)$ by using random number, and solve the linear system $AX = b$ by using Gaussian partial pivoting elimination algorithm.

3.2

There are two important points:\

- 1)Generate A nonsingular matrix to make it solvable;\
- 2)Using sort algorithm to find the partial pivot.

```

1 n=50;
2 while(1)
3     A=round(rand(n)*99)+1;    %生成n阶方阵，元素取值范围为1-100
4     %判断矩阵是否奇异
5     if d=det(A)

```



```

6         break
7     end
8 end
9 b=round(rand(n,1)*99)+1;    %生成n*1向量，元素取值范围为1-100
10 X=[n,1];
11 T=[A,b];
12 Temp=[n+1,1];    %用作交换两行
13
14 for i=1:n
15     for j=i+1:n    %选取绝对值最大的为列主元
16         if(abs(T(j,i))>abs(T(i,i)))
17             max=T(j,:);
18             Temp=T(i,:);
19             T(i,:)=T(j,:);
20             T(j,:)=Temp;
21         end
22     end
23
24     for k=i+1:n    %计算得到上三角矩阵
25         m_ik=T(k,i)/T(i,i); %计算系数
26         T(k,i+1:n+1)=T(k,i+1:n+1)-(T(i,i+1:n+1).*m_ik);
27         T(k,i)=0;
28     end
29 end
30
31 X(n,1)=T(n,n+1)/T(n,n);    %计算x[n,1]
32 for i=2:n    %计算x[1:n-1,1]
33     X(n-i+1,1)=(T(n-i+1,n+1)-T(n-i+2,n-i+2:n)*X(n-i+2:n,1))/T(i,i);
34 end

```

Randomly generate tridiagonal matrix ($30 * 30$), and use Jacobi iteration and Gauss Seidel iteration to solve them respectively. The error of the solution in terms of norm is required to be less than $1/1000$.

The most important premise to solve the problem is we can get a matrix which match the Jacobi iterative and Gauss-Seidel iterative conditions.

A strictly diagonally dominant is eligibility.

```

1    %创建30阶随机方阵，元素取值1-100
2    A=round(rand(30)*99)+1;
3    D=zeros(30);
4    U=zeros(30);

```

```

5 L=zeros(30);
6 B=round(rand(30,1)*99)+1;
7
8 %将D-U-L变为严格对角占优矩阵，使得满足迭代条件
9 for j=1:29
10     U(j,j+1)=-A(j,j+1);
11     L(j+1,j)=-A(j+1,j);
12 end
13 D(1,1)=sum(abs(A(1,1:2)))+10*rand(1);
14 for i=2:29
15     D(i,i)=sum(abs(A(i,i-1:i+1)))+10*rand(1);
16 end
17 D(30,30)=sum(abs(A(30,29:30)))+10*rand(1);
18
19 T=((D-L)^(-1))*U;
20 C=(D-L)^(-1);
21
22 e=max(abs(eig(T))); %检查系数矩阵特征值
23
24 Xn=zeros(30,1); %存储当前的迭代结果
25 Xn_1=ones(30,1); %存储上一次的迭代结果
26 while sqrt(sum(abs(Xn-Xn_1).^2))>0.001
27     Xn_1=Xn;
28     for j=1:30
29         Xn(j,1)=T(j,:)*Xn+C(j,:)*B;
30     end
31     fprintf('Xn = %f\n',Xn);
32 end

1 %创建30阶随机方阵，元素取值1-100
2 A=round(rand(30)*99)+1;
3 D=zeros(30);
4 U=zeros(30);
5 L=zeros(30);
6 B=round(rand(30,1)*99)+1;
7
8 %将D-U-L变为对角占优矩阵，使得满足迭代条件
9 for j=1:29
10     U(j,j+1)=-A(j,j+1);
11     L(j+1,j)=-A(j+1,j);
12 end
13 D(1,1)=sum(abs(A(1,1:2)))+10*rand(1);
14 for i=2:29

```

```

15     D(i,i)=sum(abs(A(i,i-1:i+1)))+10*rand(1);
16 end
17 D(30,30)=sum(abs(A(30,29:30)))+10*rand(1);
18
19 T=(D^-1)*(L+U);
20 e=max(abs(eig(T)));    %检查系数矩阵特征值
21
22 Xn=zeros(30,1);    %存储当前的迭代结果
23 Xn_1=ones(30,1);    %存储上一次的迭代结果
24 while sqrt(sum(abs(Xn-Xn_1).^2))>0.001
25     Xn_1=Xn;
26     Xn=(D^-1)*(L+U)*Xn_1+(D^-1)*B;
27     fprintf(' Xn = %f\n',Xn);
28 end

```

Question 4

Use appropriate polynomial interpolation to approximate the following ellipse.

(1) $x^2 + y^2 = 5^2$;

(2) $\rho(\theta) = 2(1 - \sin\theta)$ (心形线).

It's useful that using Spline Interpolation for the curve which is differentiable. But

(1) There are some point whose derivative are infinity for $f(x)$. We can divide the curve into several parts which includes one special point at least. And exchanging independent variable and dependent variable make some special points differentiable.

(2) Some points are nondifferentiable but continuous, they can be the endpoints.

```

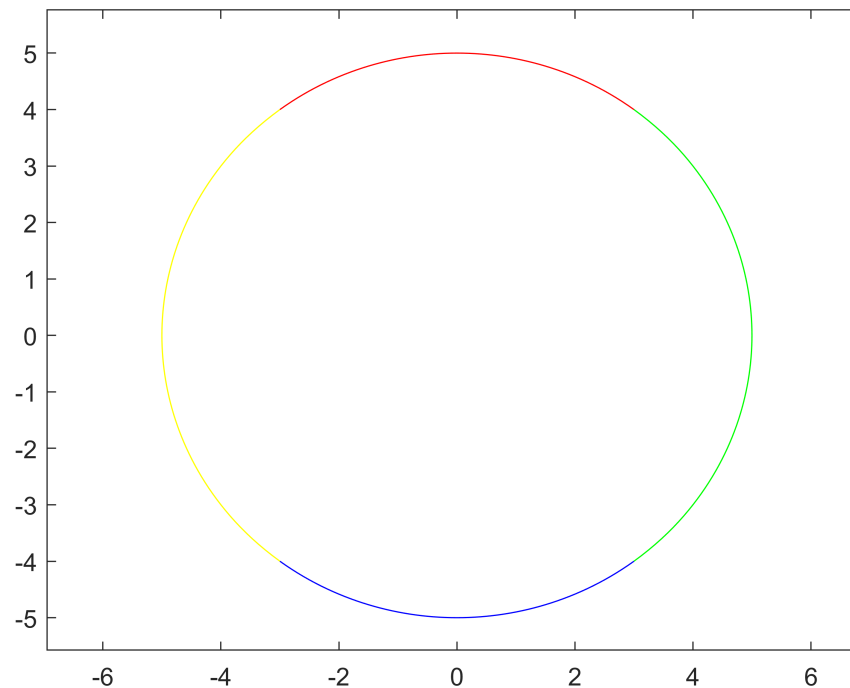
1  %对上下两段插值
2  x_i=-3:0.5:3;
3  y1=(25-x_i.^2).^(1/2);
4  y2=-(25-x_i.^2).^(1/2);
5  xx_i=-3:0.1:3;
6  yy1=spline(x_i,y1,xx_i);
7  yy2=spline(x_i,y2,xx_i);
8  %对左右两段插值
9  y_i=-4:0.5:4;
10 x1=(25-y_i.^2).^(1/2);
11 x2=-(25-y_i.^2).^(1/2);
12 yy_i=-4:0.1:4;
13 xx1=spline(y_i,x1,yy_i);

```

```

14 xx2=spline(y_i, x2, yy_i);
15 %画出图像
16 plot(xx_i, yy1, xx_i, yy2, xx1, yy_i, xx2, yy_i);

```



```

1  theta_1=0:pi/36:pi/2;
2  x=rho(theta_1).*cos(theta_1);
3  y=rho(theta_1).*sin(theta_1);
4  theta_11=0:pi/108:pi/2;
5  xx_1=rho(theta_11).*cos(theta_11);
6  yy_1=spline(x, y, xx_1);
7
8  theta_2=pi/2:pi/36:pi;
9  x=rho(theta_2).*cos(theta_2);
10 y=rho(theta_2).*sin(theta_2);
11 theta_22=pi/2:pi/108:pi;
12 xx_2=rho(theta_22).*cos(theta_22);
13 yy_2=spline(x, y, xx_2);
14
15 theta_3=pi:pi/36:(8*pi)/6;
16 x=rho(theta_3).*cos(theta_3);
17 y=rho(theta_3).*sin(theta_3);
18 theta_33=pi:pi/108:(8*pi)/6;
19 yy_3=rho(theta_33).*sin(theta_33);
20 xx_3=spline(y, x, yy_3);
21

```

```

22 theta_4=(8*pi)/6:pi/36:(10*pi)/6;
23 x=rho(theta_4).*cos(theta_4);
24 y=rho(theta_4).*sin(theta_4);
25 theta_44=(8*pi)/6:pi/108:(10*pi)/6;
26 xx_4=rho(theta_44).*cos(theta_44);
27 yy_4=spline(x, y, xx_4);
28
29 theta_5=(10*pi)/6:pi/36:2*pi;
30 x=rho(theta_5).*cos(theta_5);
31 y=rho(theta_5).*sin(theta_5);
32 theta_55=(10*pi)/6:pi/108:2*pi;
33 yy_5=rho(theta_55).*sin(theta_55);
34 xx_5=spline(y, x, yy_5);
35
36 plot(xx_1, yy_1, xx_2, yy_2, xx_3, yy_3, xx_4, yy_4, xx_5, yy_5);
37 function rho=rho(theta)
38 rho=2.*(1-sin(theta));
39 end

```



Question 5

方程術曰，置上禾三秉，中禾二秉，下禾一秉，實三十九斗，於右方。中、左禾列如右方。以右行上禾遍乘中行而以直除。又乘其次，亦以直除。然以中行中禾不盡者遍乘左行而以直除。左方下禾不盡者，上為法，下為實。實即下禾之實。求中禾，以法乘中行下實，而除下禾之實。

餘如中禾秉數而一，即中禾之實。求上禾亦以法乘右行下實，而除下禾、中禾之實。餘如上禾秉數而一，即上禾之實。實皆如法，各得一斗。

原始矩阵如下：

1	2	3
2	3	2
3	1	1
26	34	39

《九章算术》里记载的第一步变换是“以 右行上禾，遍乘中行”，也就是用右列第一项的数字 3，去乘中间那列的每一项。乘过以后，原始矩阵变换如下：

1	6	3
2	9	2
3	3	1
26	102	39

然后让中列每一项减去右列对应项的某个常数倍（这里取 2 倍），矩阵变换成：

1	0	3
2	5	2
3	1	1
26	24	39

然后“又乘其次，亦以直除”，将左边那列也乘以某个常数（这里乘以 3），让左列减右列，得到：

0	0	3
4	5	2
8	1	1
39	24	39

然后“以中行中禾不尽者遍乘左行，而以直除”，让左列乘以中列未消去的中间项5，再减去中列各项的某个常数倍（这里取4 倍）得到：

0	0	3
0	5	2
36	1	1
99	24	39

经过以上四步变换，左列数字出现了两个零，相当于消去了两个未知数，只剩下36和99，相当于 $36z=99$ 。99除以36，得到 $z=2.75$ 。

沿用前面的变换方法继续消元，并代入求解，得到 $x=9.25$ ， $y=4.25$ ，方程组被完整求解。

This method is Gaussian Elimination Method